

CAO ASSIGNMENT 4

24MAI0065 NITHISH

- 1) To perform the multiplication of two matrix using parallel constructs and compare parallel and serial time.

PROGRAM CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void multiply_serial(int **A, int **B, int **C, int N) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            C[i][j] = 0;
            for (int k = 0; k < N; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

void multiply_parallel(int **A, int **B, int **C, int N) {
    #pragma omp parallel for collapse(2)
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            C[i][j] = 0;
            for (int k = 0; k < N; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

int main() {
    int N = 1000;
    int **A = (int **)malloc(N * sizeof(int *));
    int **B = (int **)malloc(N * sizeof(int *));
    int **C = (int **)malloc(N * sizeof(int *));
    for (int i = 0; i < N; i++) {
        A[i] = (int *)malloc(N * sizeof(int));
        B[i] = (int *)malloc(N * sizeof(int));
        C[i] = (int *)malloc(N * sizeof(int));
    }
}
```

```

// Initialize matrices A and B
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        A[i][j] = i + j;
        B[i][j] = i - j;
    }
}

double start_time, end_time;

// Serial Multiplication
start_time = omp_get_wtime();
multiply_serial(A, B, C, N);
end_time = omp_get_wtime();
printf("Serial Multiplication Time: %f seconds\n", end_time - start_time);

// Parallel Multiplication
start_time = omp_get_wtime();
multiply_parallel(A, B, C, N);
end_time = omp_get_wtime();
printf("Parallel Multiplication Time: %f seconds\n", end_time - start_time);

// Free memory
for (int i = 0; i < N; i++) {
    free(A[i]);
    free(B[i]);
    free(C[i]);
}
free(A);
free(B);
free(C);

return 0;
}

```

PROGRAM SCREENSHOT:

```
nithish@nithish-VirtualBox: -
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void multiply_serial(int **A, int **B, int **C, int N) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            C[i][j] = 0;
            for (int k = 0; k < N; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

void multiply_parallel(int **A, int **B, int **C, int N) {
    #pragma omp parallel for collapse(2)
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            C[i][j] = 0;
            for (int k = 0; k < N; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

int main() {
    int N = 1000;
    int **A = (int **)malloc(N * sizeof(int *));
    int **B = (int **)malloc(N * sizeof(int *));
    int **C = (int **)malloc(N * sizeof(int *));
    for (int i = 0; i < N; i++) {
        A[i] = (int *)malloc(N * sizeof(int));
        B[i] = (int *)malloc(N * sizeof(int));
        C[i] = (int *)malloc(N * sizeof(int));
    }

    // Initialize matrices A and B
}
```

1,1 Top

```
nithish@nithish-VirtualBox: -
    B[i] = (int *)malloc(N * sizeof(int));
    C[i] = (int *)malloc(N * sizeof(int));
}

// Initialize matrices A and B
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        A[i][j] = i + j;
        B[i][j] = i - j;
    }
}

double start_time, end_time;

// Serial Multiplication
start_time = omp_get_wtime();
multiply_serial(A, B, C, N);
end_time = omp_get_wtime();
printf("Serial Multiplication Time: %f seconds\n", end_time - start_time);

// Parallel Multiplication
start_time = omp_get_wtime();
multiply_parallel(A, B, C, N);
end_time = omp_get_wtime();
printf("Parallel Multiplication Time: %f seconds\n", end_time - start_time);

// Free memory
for (int i = 0; i < N; i++) {
    free(A[i]);
    free(B[i]);
    free(C[i]);
}
free(A);
free(B);
free(C);

return 0;
}
```

73,0-1 Bot

PROGRAM OUTPUT:

```
nithish@nithish-VirtualBox: $ vi 24mat0065_multiplication.c
nithish@nithish-VirtualBox: $ gcc -fopenmp -o 24mat0065_multiplication 24mat0065_multiplication.c
nithish@nithish-VirtualBox: $ ./24mat0065_multiplication
Serial Multiplication Time: 7.186513 seconds
Parallel Multiplication Time: 6.130155 seconds
nithish@nithish-VirtualBox: $
```

2. Parallelize the Jacobi Method. Analyze the speedup and efficiency of the parallelized code. Vary the size of your A matrix and measure the runtime with one thread. For each matrix size, change the number of threads from 2,4,8, ... and plot the speedup versus the number of threads. Compute the efficiency. Explain whether or not the scaling behavior is as expected

PROGRAM CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <math.h>

#include <omp.h>

#define TOLERANCE 1e-6 // Stopping condition tolerance

void jacobi_parallel(int N, double **A, double *b, double *x, int max_iter, int num_threads) {
    double *x_new = (double *)malloc(N * sizeof(double));
    int iter = 0;
    double error;

    omp_set_num_threads(num_threads); // Set number of OpenMP threads

    do {
        error = 0.0;
        // Parallelize the main loop
        #pragma omp parallel for reduction(+:error)
        for (int i = 0; i < N; i++) {
            double sigma = 0.0;
            for (int j = 0; j < N; j++) {
                if (j != i) {
                    sigma += A[i][j] * x[j];
                }
            }
            x_new[i] = (b[i] - sigma) / A[i][i];
        }
        error = max_error(x, x_new, N);
        iter++;
    } while (error > TOLERANCE && iter < max_iter);

    free(x_new);
}
```

```

        }
    }
    x_new[i] = (b[i] - sigma) / A[i][i];
    error += fabs(x_new[i] - x[i]);
}

// Copy x_new to x for the next iteration
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    x[i] = x_new[i];
}

iter++;
} while (error > TOLERANCE && iter < max_iter);

free(x_new);
}

int main() {
    int N, max_iter = 10000;
    printf("Enter matrix size N: ");
    scanf("%d", &N);

    // Allocate memory for A, b, and x
    double **A = (double **)malloc(N * sizeof(double *));
    for (int i = 0; i < N; i++) {
        A[i] = (double *)malloc(N * sizeof(double));
    }
    double *b = (double *)malloc(N * sizeof(double));
    double *x = (double *)calloc(N, sizeof(double)); // Initial guess is 0

```

```

// Initialize matrix A and vector b with random values (diagonally dominant)
srand(42); // For reproducibility
for (int i = 0; i < N; i++) {
    double row_sum = 0.0;
    for (int j = 0; j < N; j++) {
        A[i][j] = rand() % 10 + 1;
        if (i != j) row_sum += fabs(A[i][j]);
    }
    A[i][i] = row_sum + rand() % 10 + 1; // Ensure diagonal dominance
    b[i] = rand() % 10 + 1;
}

// Measure runtime for different thread counts
int num_threads[] = {1, 2, 4, 8, 16}; // Adjust based on available cores
for (int t = 0; t < 5; t++) {
    // Reset x to the initial guess (all zeros) before each run
    for (int i = 0; i < N; i++) {
        x[i] = 0.0;
    }

    double start = omp_get_wtime();
    jacobi_parallel(N, A, b, x, max_iter, num_threads[t]);
    double end = omp_get_wtime();
    printf("Threads: %d, Time: %f seconds\n", num_threads[t], end - start);
}

// Free memory
for (int i = 0; i < N; i++) {
    free(A[i]);
}

```

```

    }

    free(A);

    free(b);

    free(x);

    return 0;
}

```

PROGRAM CODE SCREENSHOT:

```

nithish@nithish-VirtualBox: ~
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define TOLERANCE 1e-6 // Stopping condition tolerance

void jacobi_parallel(int N, double **A, double *b, double *x, int max_iter, int num_threads) {
    double *x_new = (double *)malloc(N * sizeof(double));
    int iter = 0;
    double error;

    omp_set_num_threads(num_threads); // Set number of OpenMP threads

    do {
        error = 0.0;
        // Parallelize the main loop
        #pragma omp parallel for reduction(+:error)
        for (int i = 0; i < N; i++) {
            double sigma = 0.0;
            for (int j = 0; j < N; j++) {
                if (j != i) {
                    sigma += A[i][j] * x[j];
                }
            }
            x_new[i] = (b[i] - sigma) / A[i][i];
            error += fabs(x_new[i] - x[i]);
        }

        // Copy x_new to x for the next iteration
        #pragma omp parallel for
        for (int i = 0; i < N; i++) {
            x[i] = x_new[i];
        }

        iter++;
    } while (error > TOLERANCE && iter < max_iter);

    free(x_new);
}

```

```

nithish@nithish-VirtualBox: ~
}

free(x_new);
}

int main() {
    int N, max_iter = 10000;
    printf("Enter matrix size N: ");
    scanf("%d", &N);

    // Allocate memory for A, b, and x
    double **A = (double **)malloc(N * sizeof(double *));
    for (int i = 0; i < N; i++) {
        A[i] = (double *)malloc(N * sizeof(double));
    }
    double *b = (double *)malloc(N * sizeof(double));
    double *x = (double *)calloc(N, sizeof(double)); // Initial guess is 0

    // Initialize matrix A and vector b with random values (diagonally dominant)
    srand(42); // For reproducibility
    for (int i = 0; i < N; i++) {
        double row_sum = 0.0;
        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 10 + 1;
            if (i != j) row_sum += fabs(A[i][j]);
        }
        A[i][i] = row_sum + rand() % 10 + 1; // Ensure diagonal dominance
        b[i] = rand() % 10 + 1;
    }

    // Measure runtime for different thread counts
    int num_threads[] = {1, 2, 4, 8, 16}; // Adjust based on available cores
    for (int t = 0; t < 5; t++) {
        // Reset x to the initial guess (all zeros) before each run
        for (int i = 0; i < N; i++) {
            x[i] = 0.0;
        }

        double start = omp_get_wtime();
        jacobi_parallel(N, A, b, x, max_iter, num_threads[t]);
        double end = omp_get_wtime();
    }
}

```



```
nithish@nithish-VirtualBox: ~  
double *x = (double *)calloc(N, sizeof(double)); // Initial guess is 0  
  
// Initialize matrix A and vector b with random values (diagonally dominant)  
srand(42); // For reproducibility  
for (int i = 0; i < N; i++) {  
    double row_sum = 0.0;  
    for (int j = 0; j < N; j++) {  
        A[i][j] = rand() % 10 + 1;  
        if (i != j) row_sum += fabs(A[i][j]);  
    }  
    A[i][i] = row_sum + rand() % 10 + 1; // Ensure diagonal dominance  
    b[i] = rand() % 10 + 1;  
}  
  
// Measure runtime for different thread counts  
int num_threads[] = {1, 2, 4, 8, 16}; // Adjust based on available cores  
for (int t = 0; t < 5; t++) {  
    // Reset x to the initial guess (all zeros) before each run  
    for (int i = 0; i < N; i++) {  
        x[i] = 0.0;  
    }  
  
    double start = omp_get_wtime();  
    jacobi_parallel(N, A, b, x, max_iter, num_threads[t]);  
    double end = omp_get_wtime();  
    printf("Threads: %d, Time: %f seconds\n", num_threads[t], end - start);  
}  
  
// Free memory  
for (int i = 0; i < N; i++) {  
    free(A[i]);  
}  
free(A);  
free(b);  
free(x);  
  
return 0;  
}
```

91,0-1 Bot

PROGRAM OUTPUT:

```
nithish@nithish-VirtualBox: ~  
nithish@nithish-VirtualBox: $ vi 24mai0065_chlexp.c  
nithish@nithish-VirtualBox: $ gcc -fopenmp -o 24mai0065_chlexp 24mai0065_chlexp.c  
nithish@nithish-VirtualBox: $ ./24mai0065_chlexp  
Enter matrix size N: 1000  
Threads: 1, Time: 31.406420 seconds  
Threads: 2, Time: 32.040073 seconds  
Threads: 4, Time: 31.549374 seconds  
Threads: 8, Time: 33.881467 seconds  
Threads: 16, Time: 32.449023 seconds  
nithish@nithish-VirtualBox: $
```

GRAPH REPRESENTATION:

